

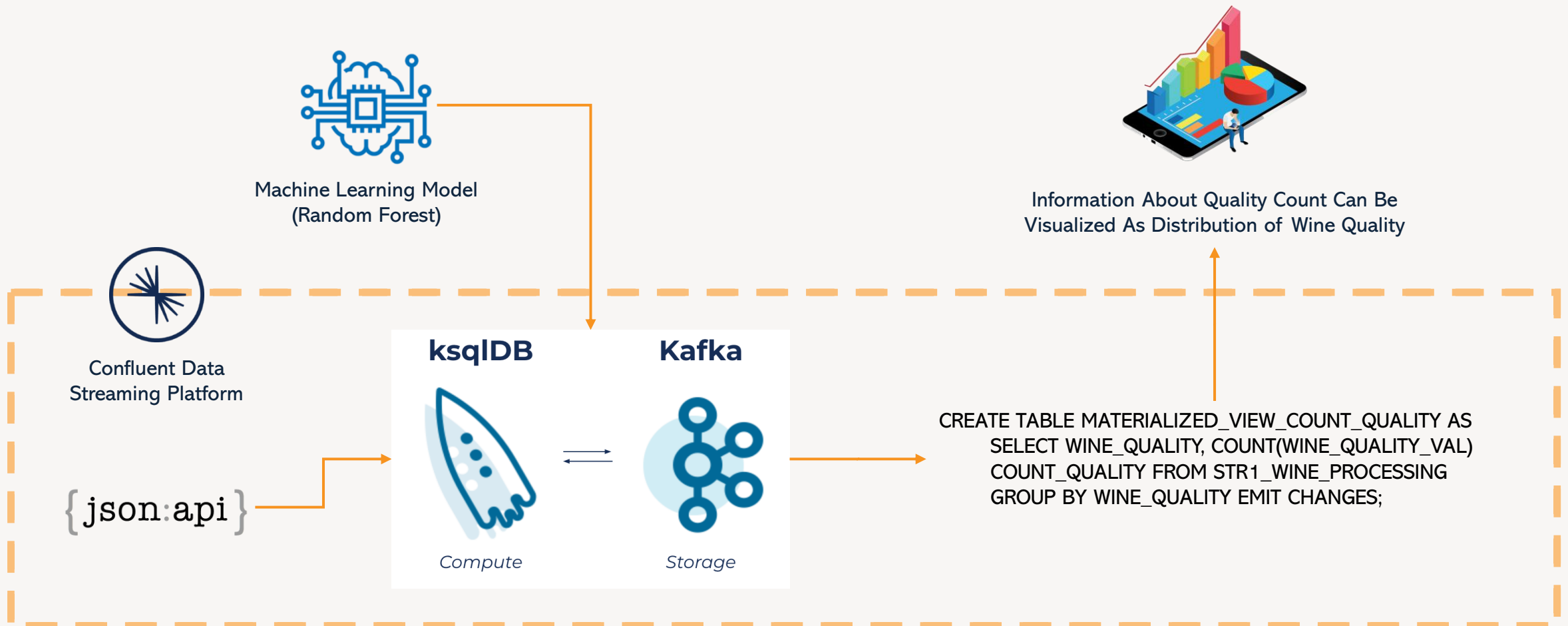
Stream Processing + Machine Learning: Event Driven Wine Quality Prediction

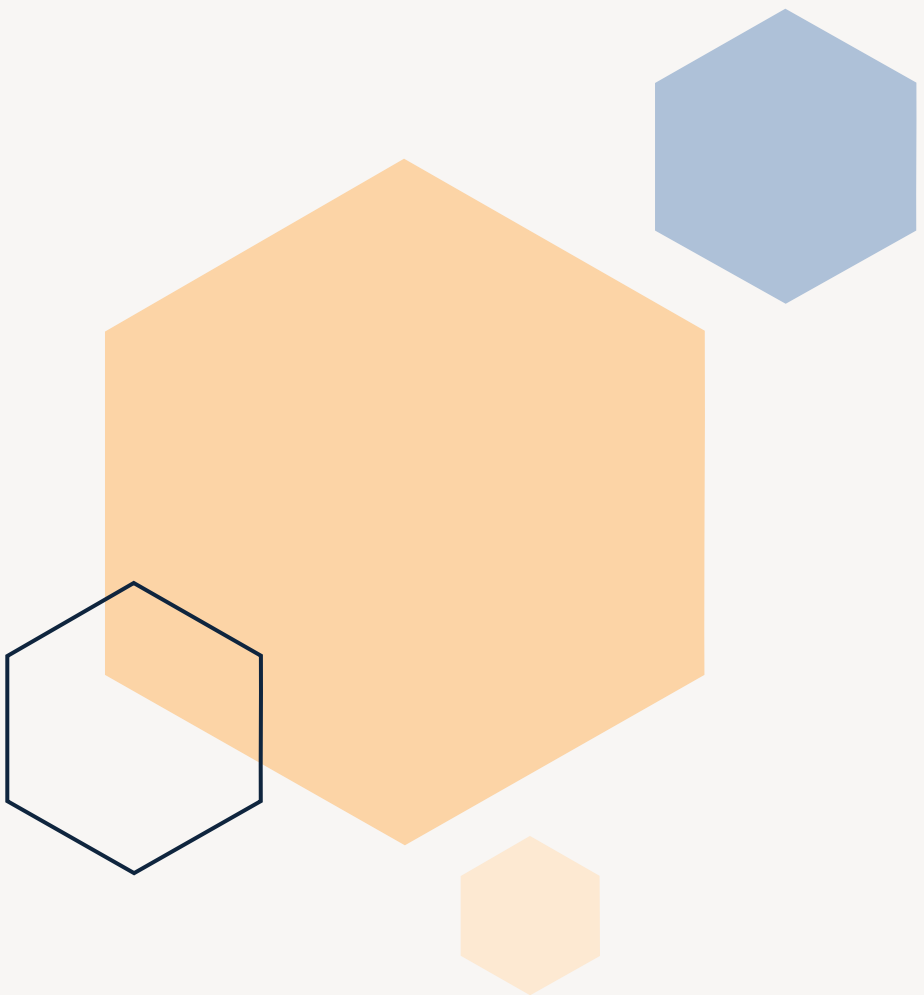
Yunata Gunawan

April 21, 2024



System Integration





Dataset + Machine Learning Model from:

<https://www.kaggle.com/datasets/yasserh/wine-quality-dataset>

<https://www.kaggle.com/code/ramezmohamed/wine-quality-extratreesclassifier-83-acc>

Stream Input, Output Data

[Machine Learning Sample Wine Quality Dataset]

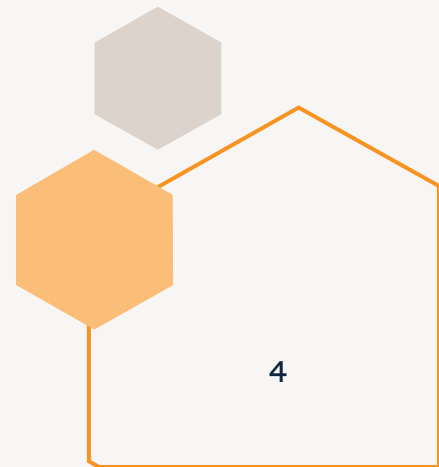
```
{  
  "wine id": 1,  
  "fixed acidity": 7.3,  
  "volatile acidity": 0.65,  
  "citric acid": 0.0,  
  "residual sugar": 1.2,  
  "chlorides": 0.065,  
  "free sulfur dioxide": 15.0,  
  "total sulfur dioxide": 21.0,  
  "density": 0.9946,  
  "pH": 3.39,  
  "sulphates": 0.47,  
  "alcohol": 10.0,  
  "quality": 7  
}
```

[Input JSON]

```
{  
  "wine id": 1,  
  "fixed acidity": 7.3,  
  "volatile acidity": 0.65,  
  "citric acid": 0.0,  
  "residual sugar": 1.2,  
  "chlorides": 0.065,  
  "free sulfur dioxide": 15.1,  
  "total sulfur dioxide": 21.0,  
  "density": 0.9946,  
  "pH": 3.39,  
  "sulphates": 0.47,  
  "alcohol": 10.0  
}
```

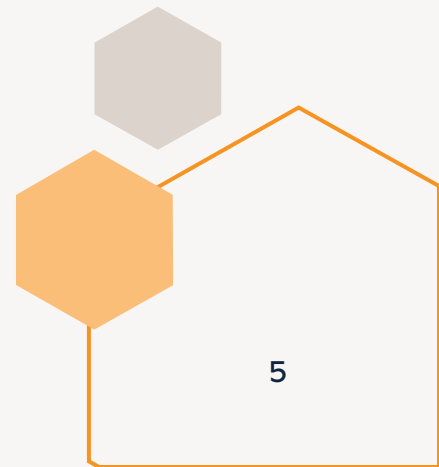
[Output Columns]

```
{  
  "wine id": 1,  
  "quality": 7  
}
```



Confluent CLI Supporting Command

```
confluent login --save
confluent logout
confluent kafka topic consume --from-beginning wine_source
confluent environment list
confluent environment <ENV>
confluent kafka cluster list
confluent kafka cluster use <ENV>
confluent api-key store --resource <ENV>
confluent api-key use <ENV>
confluent kafka --help
confluent kafka consumer group list
confluent cluster list
confluent kafka cluster list
confluent kafka cluster describe <ENV>
telnet <ENV>.<REGION>.<CLOUD>.confluent.cloud 9092
```



Consumer-Producer Code (Part 1)

```
from confluent_kafka import Producer, Consumer, KafkaError, KafkaException
import socket
import sys
import pickle
import pandas as pd
import json

CLUSTER_API_KEY = "...
CLUSTER_API_SECRET = "...
topic_name_produce = "wine_processing"
topic_name_consume = "wine_source"
consumer_group = "confluent_cli_consumer_<HASH_STRING>"
bootstrap_server = "<ENV>.<REGION>.<CLOUD>.confluent.cloud:9092"

model_path = "./wine-prediction.pkl"

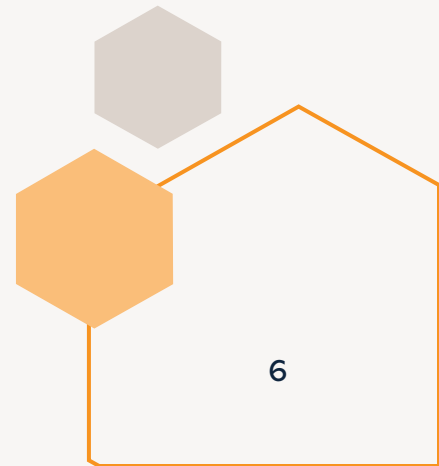
conf_producer = {'bootstrap.servers': bootstrap_server,
                 'security.protocol': 'SASL_SSL',
                 'sasl.mechanism': 'PLAIN',
                 'sasl.username': CLUSTER_API_KEY,
                 'sasl.password': CLUSTER_API_SECRET,
                 'client.id': socket.gethostname()}

conf_consumer = {'bootstrap.servers': bootstrap_server,
                 'security.protocol': 'SASL_SSL',
                 'sasl.mechanism': 'PLAIN',
                 'sasl.username': CLUSTER_API_KEY,
                 'sasl.password': CLUSTER_API_SECRET,
                 'group.id': consumer_group,
                 'auto.offset.reset': 'earliest'}

consumer = Consumer(conf_consumer)
producer = Producer(conf_producer)

with open(model_path, 'rb') as f:
    model = pickle.load(f)

running = True
```



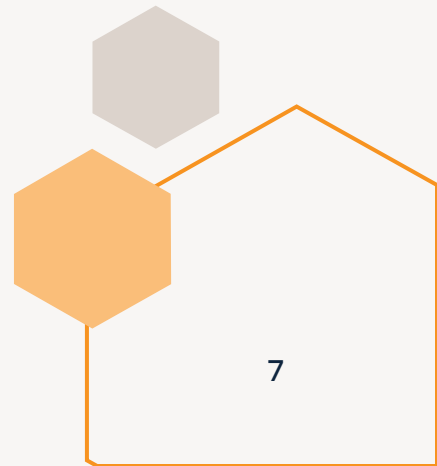
Consumer-Producer Code (Part 2)

```
def delivery_callback(err, msg):
    if err:
        print('ERROR: Message failed delivery: {}'.format(err))
    else:
        print("Produced event to topic {topic}: key = {key:12} value = {value:12}".format(
            topic=msg.topic(), key=msg.key().decode('utf-8'), value=msg.value().decode('utf-8'))))

def basic_consume_loop(consumer, topics):
    try:
        consumer.subscribe(topics)
        while running:
            msg = consumer.poll(timeout=1.0)
            if msg is None: continue
            if msg.error():
                if msg.error().code() == KafkaError._PARTITION_EOF:
                    # End of partition event
                    sys.stderr.write('%% %s [%d] reached end at offset %d\n' %
                                     (msg.topic(), msg.partition(), msg.offset()))
                elif msg.error():
                    raise KafkaException(msg.error())
            else:
                #print("Consumed event from topic {topic}: key = {key:12} value = {value:12}".format(
                #    topic=msg.topic(), key=msg.key().decode('utf-8'), value=msg.value().decode('utf-8'))))
                #print(type(msg.value().decode('utf-8'))))
                data = json.loads(msg.value().decode('utf-8'))
                df = pd.json_normalize(data)
                wine_id = df['wine id'].iloc[0]
                df = df.drop('wine id', axis=1)
                wine_quality = model.predict(df)[0]
                print(wine_id, wine_quality)
                producer.produce(topic_name_produce, key=str(wine_id), value=str(wine_quality), callback=delivery_callback)
                producer.poll(1)
        finally:
            # Close down consumer to commit final offsets.
            consumer.close()

def shutdown():
    running = False

basic_consume_loop(consumer=consumer, topics=[topic_name_consume])
```



Wine Quality Dataset

kaggle

+

Create

Home

Competitions

Datasets

Models

Code

Discussions

Learn

More

View Active Events

Search

Wine Quality Dataset

621

New Notebook

Download (22 kB)

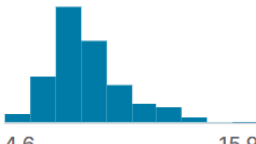
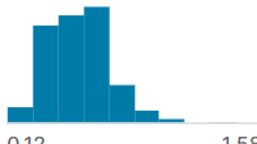
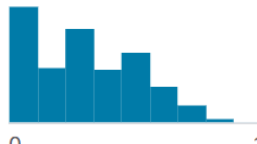
Data Card

Code (338)

Discussion (2)

Suggestions (0)

Output variable (based on sensory data):\n\n12 - quality (score between 0 and 10)

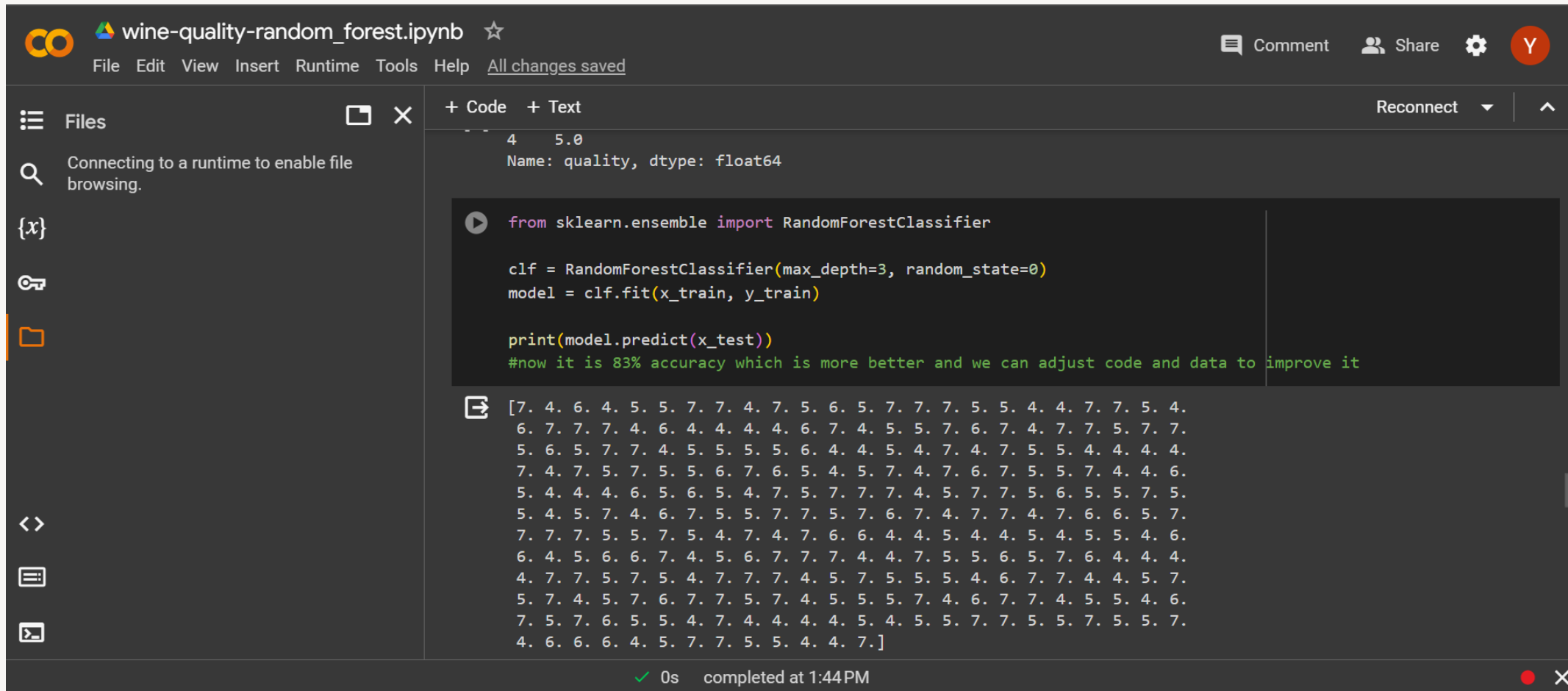
# fixed acidity	# volatile acidity	# citric acid	# residual sugar
Fixed acidity value	Volatile acidity value	Citric acid value	Residual sugar value
			
4.6	0.12	0	0.9
7.4	0.7	0.0	1.9
7.8	0.88	0.0	2.6

Summary

1 file

13 columns

Train The Model



The screenshot shows a Jupyter Notebook titled "wine-quality-random_forest.ipynb". The interface includes a top bar with "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help" menus, along with a "Comment" button, a "Share" button, and a user profile icon labeled "Y". A sidebar on the left shows a "Files" panel with a message "Connecting to a runtime to enable file browsing." and a "Code" panel with a "Reconnect" button. The main area displays the following code:

```
4 5.0
   Name: quality, dtype: float64

from sklearn.ensemble import RandomForestClassifier

clf = RandomForestClassifier(max_depth=3, random_state=0)
model = clf.fit(x_train, y_train)

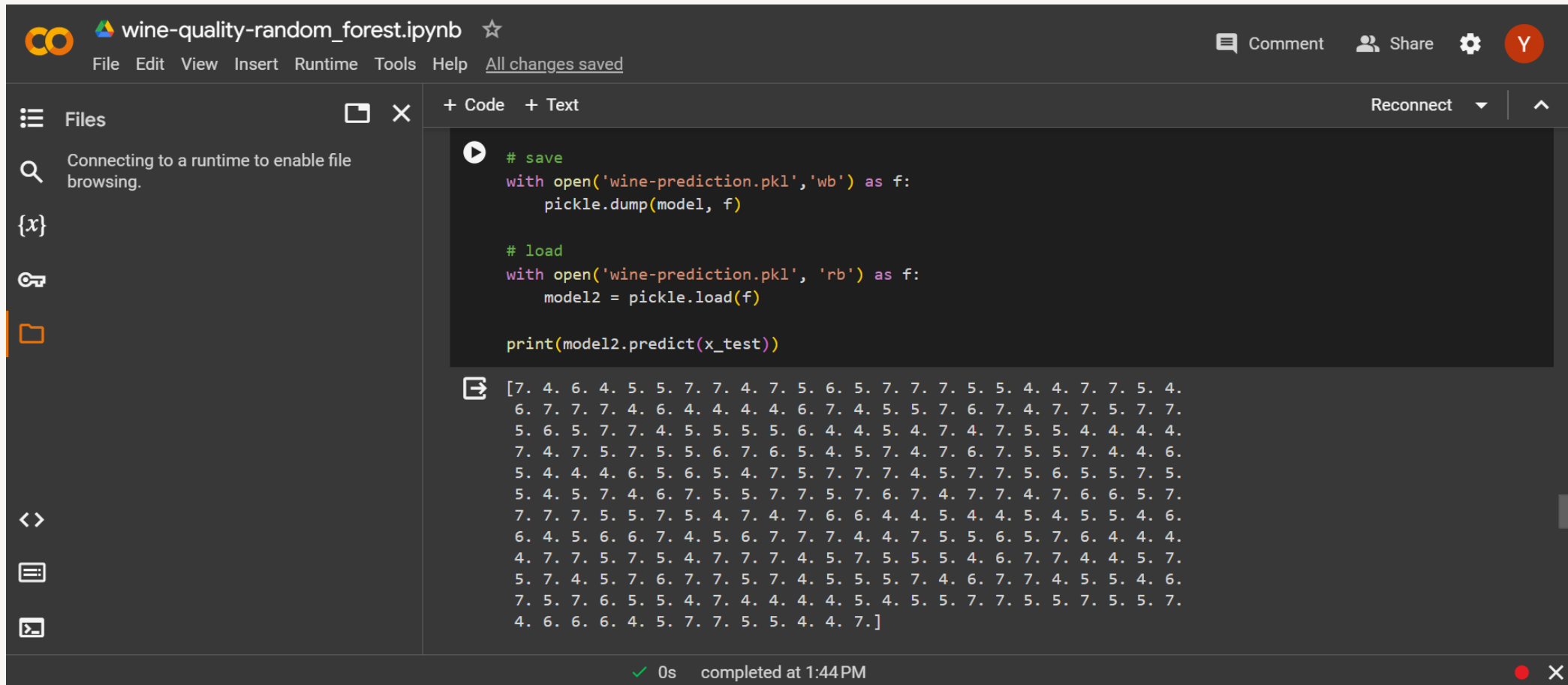
print(model.predict(x_test))
#now it is 83% accuracy which is more better and we can adjust code and data to improve it
```

The output of the code is a 10x10 grid of numerical values representing the predicted quality scores for the test set:

```
[7. 4. 6. 4. 5. 5. 7. 7. 4. 7. 5. 6. 5. 7. 7. 7. 5. 5. 4. 4. 7. 7. 5. 4.
 6. 7. 7. 7. 4. 6. 4. 4. 4. 4. 6. 7. 4. 5. 5. 7. 6. 7. 4. 7. 7. 5. 7. 7.
 5. 6. 5. 7. 7. 4. 5. 5. 5. 5. 6. 4. 4. 5. 4. 7. 4. 7. 5. 5. 4. 4. 4. 4.
 7. 4. 7. 5. 7. 5. 5. 6. 7. 6. 5. 4. 5. 7. 4. 7. 6. 7. 5. 5. 7. 4. 4. 6.
 5. 4. 4. 4. 6. 5. 6. 5. 4. 7. 5. 7. 7. 7. 4. 5. 7. 7. 5. 6. 5. 5. 7. 5.
 5. 4. 5. 7. 4. 6. 7. 5. 5. 7. 7. 5. 7. 6. 7. 4. 7. 7. 4. 7. 6. 6. 5. 7.
 7. 7. 7. 5. 5. 7. 5. 4. 7. 4. 7. 6. 6. 4. 4. 5. 4. 4. 5. 4. 5. 5. 4. 6.
 6. 4. 5. 6. 6. 7. 4. 5. 6. 7. 7. 7. 4. 4. 7. 5. 5. 6. 5. 7. 6. 4. 4. 4.
 4. 7. 7. 5. 7. 5. 4. 7. 7. 7. 4. 5. 7. 5. 5. 5. 4. 6. 7. 7. 4. 4. 5. 7.
 5. 7. 4. 5. 7. 6. 7. 7. 5. 7. 4. 4. 5. 5. 7. 4. 6. 7. 7. 4. 5. 5. 4. 6.
 7. 5. 7. 6. 5. 5. 4. 7. 4. 4. 4. 5. 4. 5. 4. 5. 5. 7. 7. 5. 5. 7. 5. 7.
 4. 6. 6. 6. 4. 5. 7. 7. 5. 5. 4. 4. 7.]
```

The bottom status bar indicates that the code was executed successfully, with a green checkmark, "0s" execution time, and "completed at 1:44 PM".

Save The Machine Learning Model



The screenshot shows a Jupyter Notebook titled "wine-quality-random_forest.ipynb". The interface includes a top menu bar with "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help", along with a status "All changes saved". On the right, there are buttons for "Comment", "Share", and a user profile icon. The left sidebar shows a "Files" panel with a message "Connecting to a runtime to enable file browsing." and icons for file operations. The main area is split into a code editor and a console output.

```
# save
with open('wine-prediction.pkl','wb') as f:
    pickle.dump(model, f)

# load
with open('wine-prediction.pkl', 'rb') as f:
    model2 = pickle.load(f)

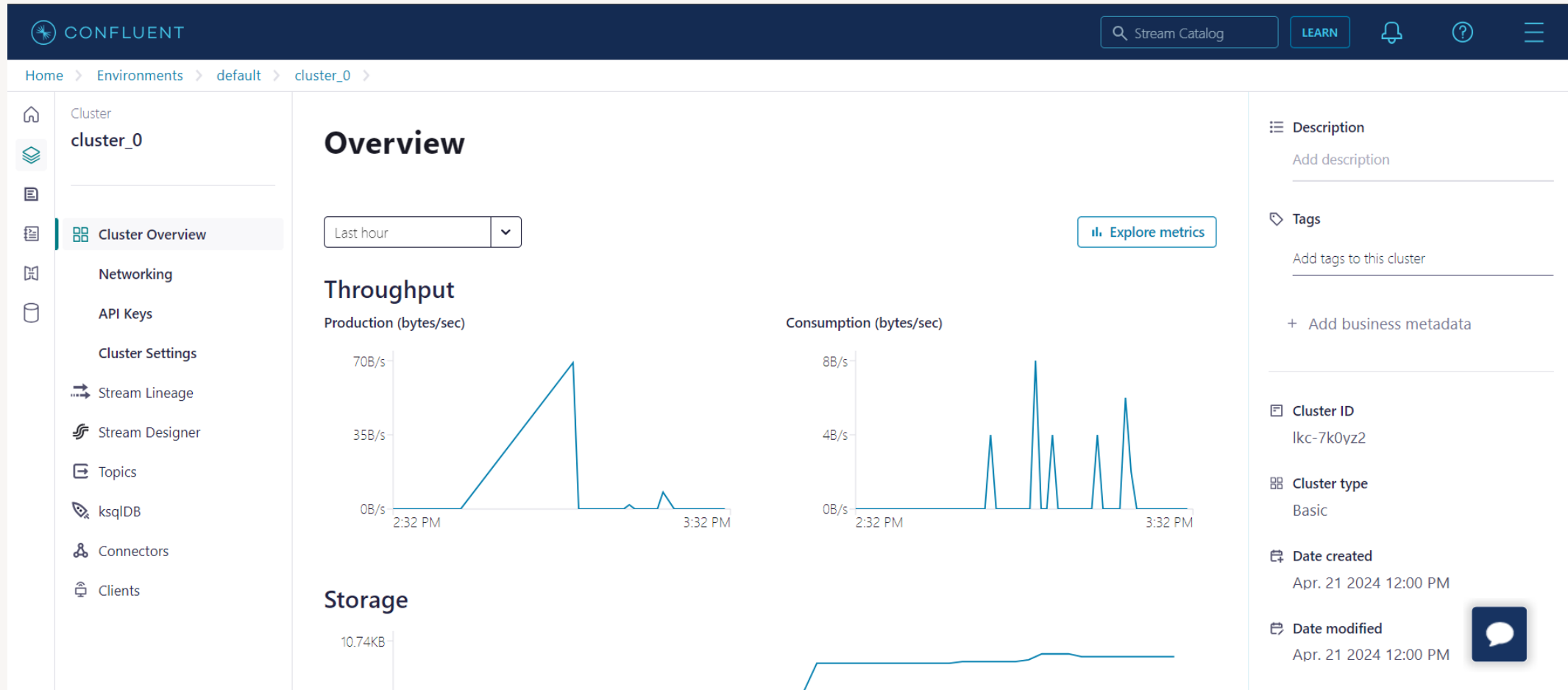
print(model2.predict(x_test))
```

The console output displays a 10x10 grid of numerical values, representing the model's predictions for a test set. The values range from 4 to 7.

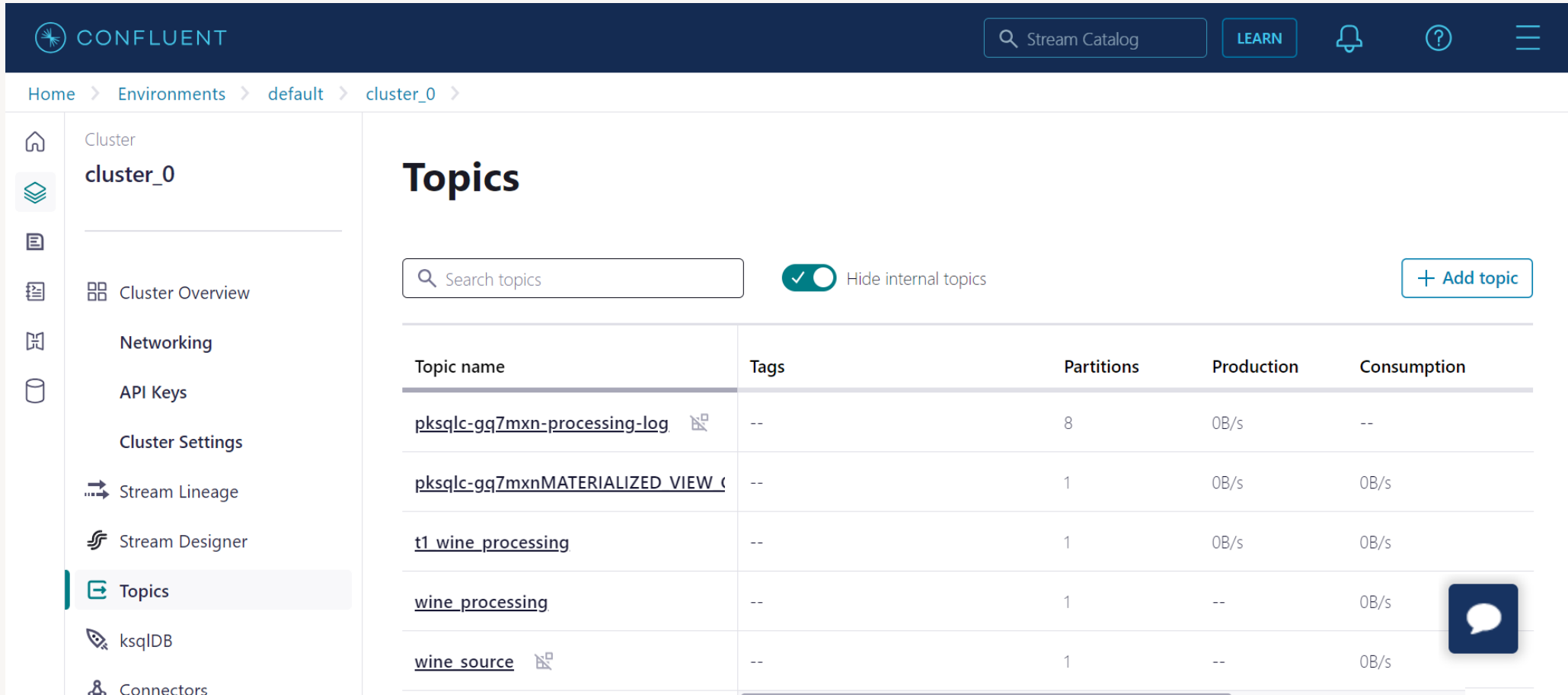
```
[7. 4. 6. 4. 5. 5. 7. 7. 4. 7. 5. 6. 5. 7. 7. 7. 5. 5. 4. 4. 7. 7. 5. 4.
 6. 7. 7. 7. 4. 6. 4. 4. 4. 4. 6. 7. 4. 5. 5. 7. 6. 7. 4. 7. 7. 5. 7. 7.
 5. 6. 5. 7. 7. 4. 5. 5. 5. 5. 6. 4. 4. 5. 4. 7. 4. 7. 5. 5. 4. 4. 4. 4.
 7. 4. 7. 5. 7. 5. 5. 6. 7. 6. 5. 4. 5. 7. 4. 7. 6. 7. 5. 5. 7. 4. 4. 6.
 5. 4. 4. 4. 6. 5. 6. 5. 4. 7. 5. 7. 7. 7. 4. 5. 7. 7. 5. 6. 5. 5. 7. 5.
 5. 4. 5. 7. 4. 6. 7. 5. 5. 7. 7. 5. 7. 6. 7. 4. 7. 7. 4. 7. 6. 6. 5. 7.
 7. 7. 7. 5. 5. 7. 5. 4. 7. 4. 7. 6. 6. 4. 4. 5. 4. 4. 5. 4. 5. 5. 4. 6.
 6. 4. 5. 6. 6. 7. 4. 5. 6. 7. 7. 7. 7. 4. 4. 7. 5. 5. 6. 5. 7. 6. 4. 4. 4.
 4. 7. 7. 5. 7. 5. 4. 7. 7. 7. 4. 5. 7. 5. 5. 5. 4. 6. 7. 7. 4. 4. 5. 7.
 5. 7. 4. 5. 7. 6. 7. 7. 5. 7. 4. 5. 5. 5. 7. 4. 6. 7. 7. 4. 5. 5. 4. 6.
 7. 5. 7. 6. 5. 5. 4. 7. 4. 4. 4. 4. 5. 4. 5. 5. 7. 7. 5. 5. 7. 5. 5. 7.
 4. 6. 6. 6. 4. 5. 7. 7. 5. 5. 4. 4. 7.]
```

The status bar at the bottom indicates the execution was successful: "✓ 0s completed at 1:44 PM".

Cluster Overview



Create Topics



The screenshot displays the Confluent web interface for managing topics in a specific cluster. The top navigation bar includes the Confluent logo, a search bar for the Stream Catalog, a 'LEARN' button, and notification and help icons. The breadcrumb trail indicates the current location: Home > Environments > default > cluster_0 > Topics.

The left sidebar contains a navigation menu for the 'cluster_0' environment, with options like Cluster Overview, Networking, API Keys, Cluster Settings, Stream Lineage, Stream Designer, Topics (highlighted), ksqlDB, and Connectors.

The main content area, titled 'Topics', features a search bar and a toggle for 'Hide internal topics' (which is currently turned on). Below this is a table listing the topics in the cluster.

Topic name	Tags	Partitions	Production	Consumption
pksqlc-gg7mxn-processing-log	--	8	0B/s	--
pksqlc-gg7mxnMATERIALIZED_VIEW_C	--	1	0B/s	0B/s
t1_wine_processing	--	1	0B/s	0B/s
wine_processing	--	1	--	0B/s
wine_source	--	1	--	0B/s

A chat bubble icon is visible in the bottom right corner of the interface.

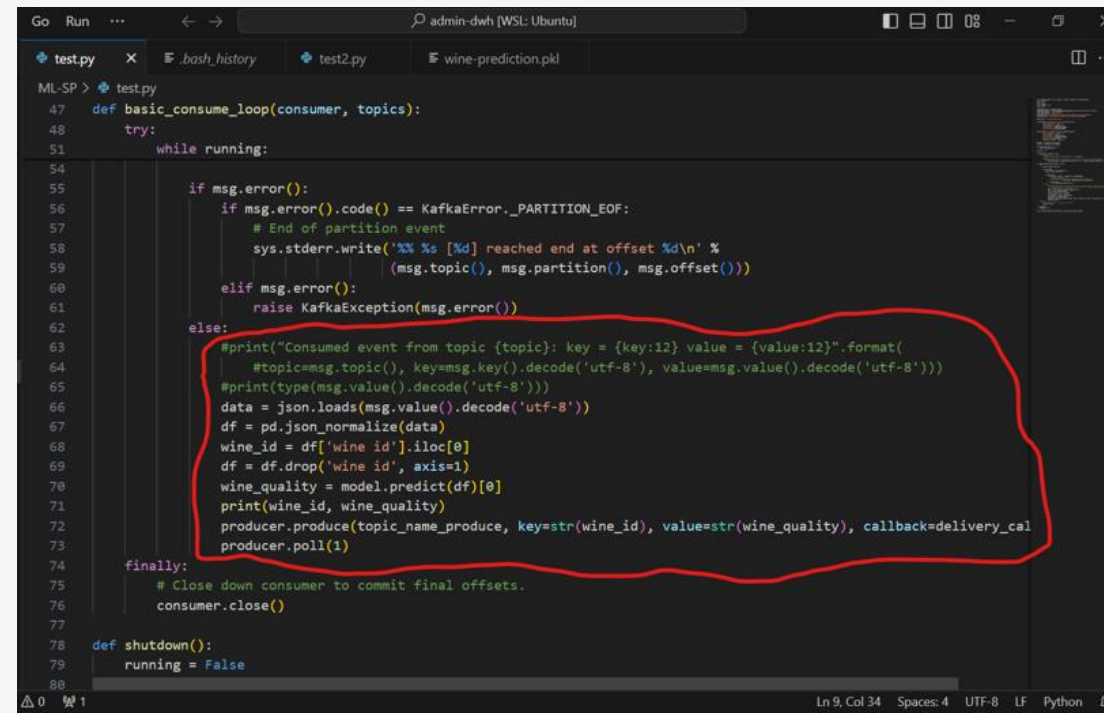
Cluster API Keys

The screenshot shows the Confluent web interface. At the top is a dark blue header with the Confluent logo, a search bar for 'Stream Catalog', a 'LEARN' button, and notification/help icons. Below the header is a breadcrumb trail: 'Home > Environments > default > cluster_0 >'. On the left is a sidebar with icons and labels for 'Cluster', 'cluster_0', 'Cluster Overview', 'Networking', 'API Keys' (highlighted), 'Cluster Settings', 'Stream Lineage', 'Stream Designer', 'Topics', 'ksqlDB', and 'Connectors'. The main content area is titled 'API keys' and contains a search bar labeled 'Search API keys' and a '+ Add key' button. Below these is a table with the following data:

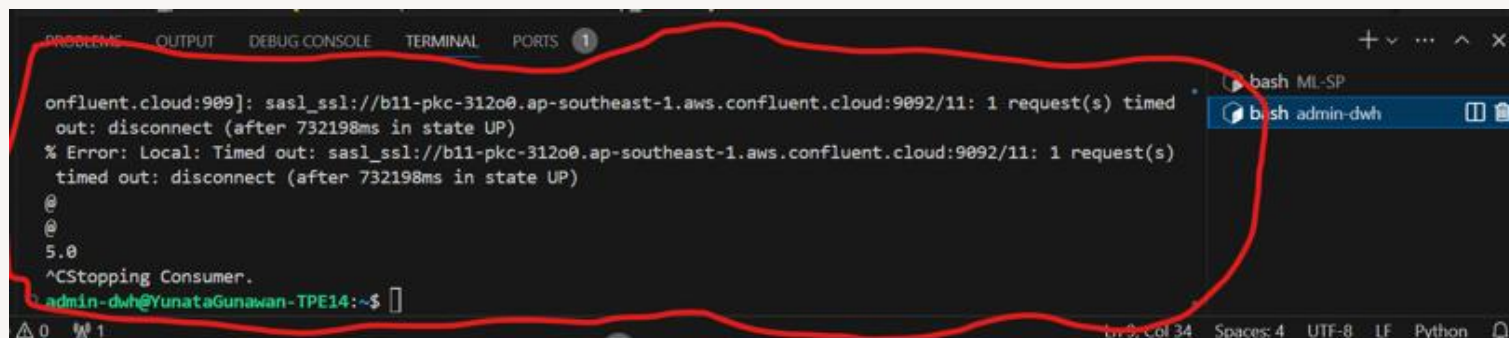
	Owner	Created	Last Modified	Description
	Student-Demo	Apr. 21 2024 12:14 PM	Apr. 21 2024 12:15 PM	python-key
	Student-Demo	Apr. 21 2024 2:09 PM	Apr. 21 2024 2:09 PM	Key for ksqlDB_cluster_0

A chat bubble icon is visible in the bottom right corner of the main content area.

ML Model Prediction Consumer Code Result



```
Go Run ... admin-dwh [WSL: Ubuntu]
test.py x .bash_history test2.py wine-prediction.pkl
ML-SP > test.py
47 def basic_consume_loop(consumer, topics):
48     try:
51         while running:
54             if msg.error():
55                 if msg.error().code() == KafkaError._PARTITION_EOF:
56                     # End of partition event
57                     sys.stderr.write('%s [%d] reached end at offset %d\n' %
58                                     (msg.topic(), msg.partition(), msg.offset()))
59                 elif msg.error():
60                     raise KafkaException(msg.error())
61             else:
62                 #print("Consumed event from topic {topic}: key = {key:12} value = {value:12}".format(
63                     #topic=msg.topic(), key=msg.key().decode('utf-8'), value=msg.value().decode('utf-8'))
64                 #print(type(msg.value().decode('utf-8')))
65                 data = json.loads(msg.value().decode('utf-8'))
66                 df = pd.json_normalize(data)
67                 wine_id = df['wine id'].iloc[0]
68                 df = df.drop('wine id', axis=1)
69                 wine_quality = model.predict(df)[0]
70                 print(wine_id, wine_quality)
71                 producer.produce(topic_name_produce, key=str(wine_id), value=str(wine_quality), callback=delivery_cal
72                 producer.poll(1)
73             finally:
74                 # Close down consumer to commit final offsets.
75                 consumer.close()
76
77 def shutdown():
78     running = False
79
80
```



```
onfluent.cloud:909]: sasl_ssl://b11-pkc-312o0.ap-southeast-1.aws.confluent.cloud:9092/11: 1 request(s) timed
out: disconnect (after 732198ms in state UP)
% Error: Local: Timed out: sasl_ssl://b11-pkc-312o0.ap-southeast-1.aws.confluent.cloud:9092/11: 1 request(s)
timed out: disconnect (after 732198ms in state UP)
@
@
5.0
^CStopping Consumer.
admin-dwh@YunataGunawan-TPE14:~$
```

Stream Processing

```
CREATE STREAM strO_wine_processing (wine_quality STRING)
  WITH (kafka_topic='wine_processing', KEY_FORMAT='KAFKA', VALUE_FORMAT='KAFKA');
```

```
SELECT * FROM strO_wine_processing EMIT CHANGES;
```

Issues: <https://github.com/confluentinc/ksql/issues/3060>

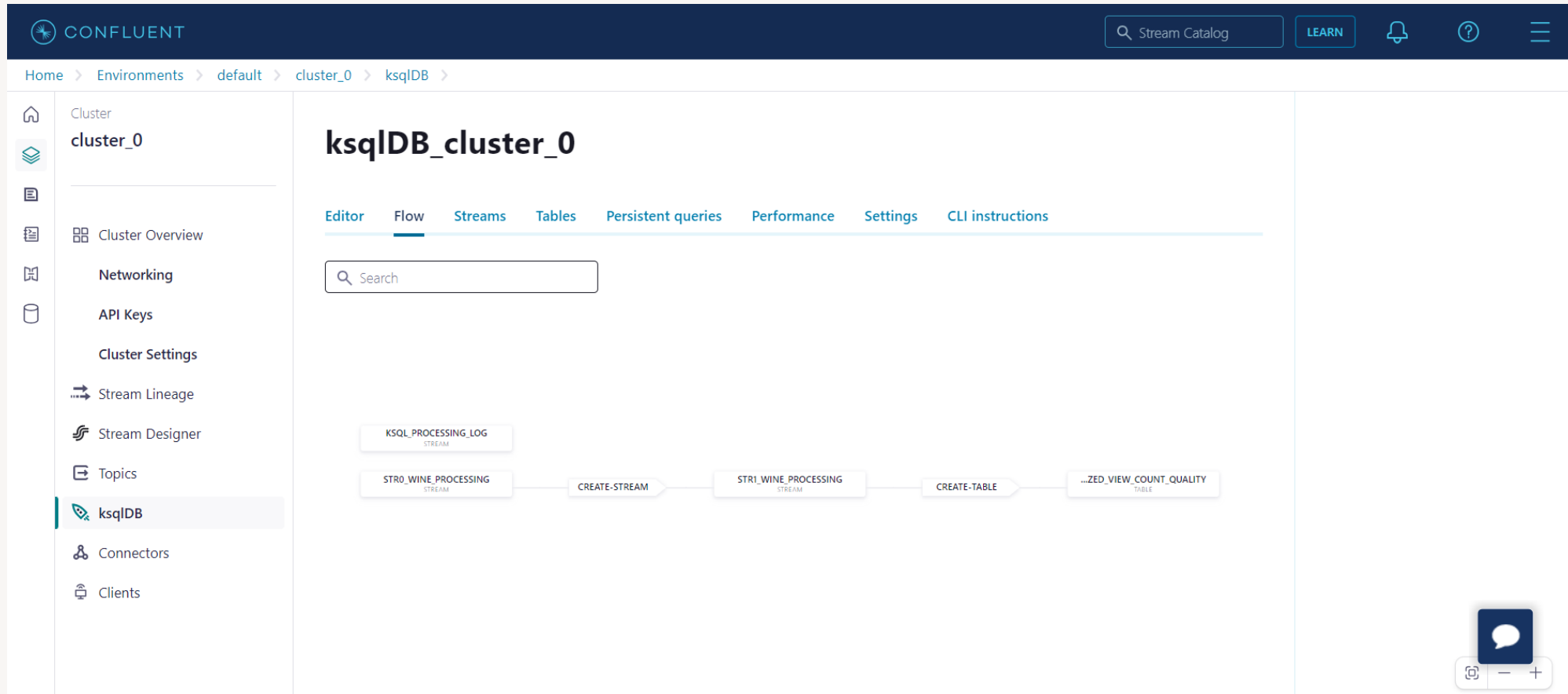
```
CREATE STREAM str1_wine_processing WITH (kafka_topic='t1_wine_processing', VALUE_FORMAT='AVRO')
AS SELECT AS_VALUE(WINE_QUALITY) WINE_QUALITY_VAL, WINE_QUALITY FROM strO_wine_processing
PARTITION BY WINE_QUALITY EMIT CHANGES;
```

```
CREATE TABLE materialized_view_count_quality AS SELECT WINE_QUALITY, COUNT(WINE_QUALITY_VAL)
COUNT_QUALITY FROM STR1_WINE_PROCESSING GROUP BY WINE_QUALITY EMIT CHANGES;
```

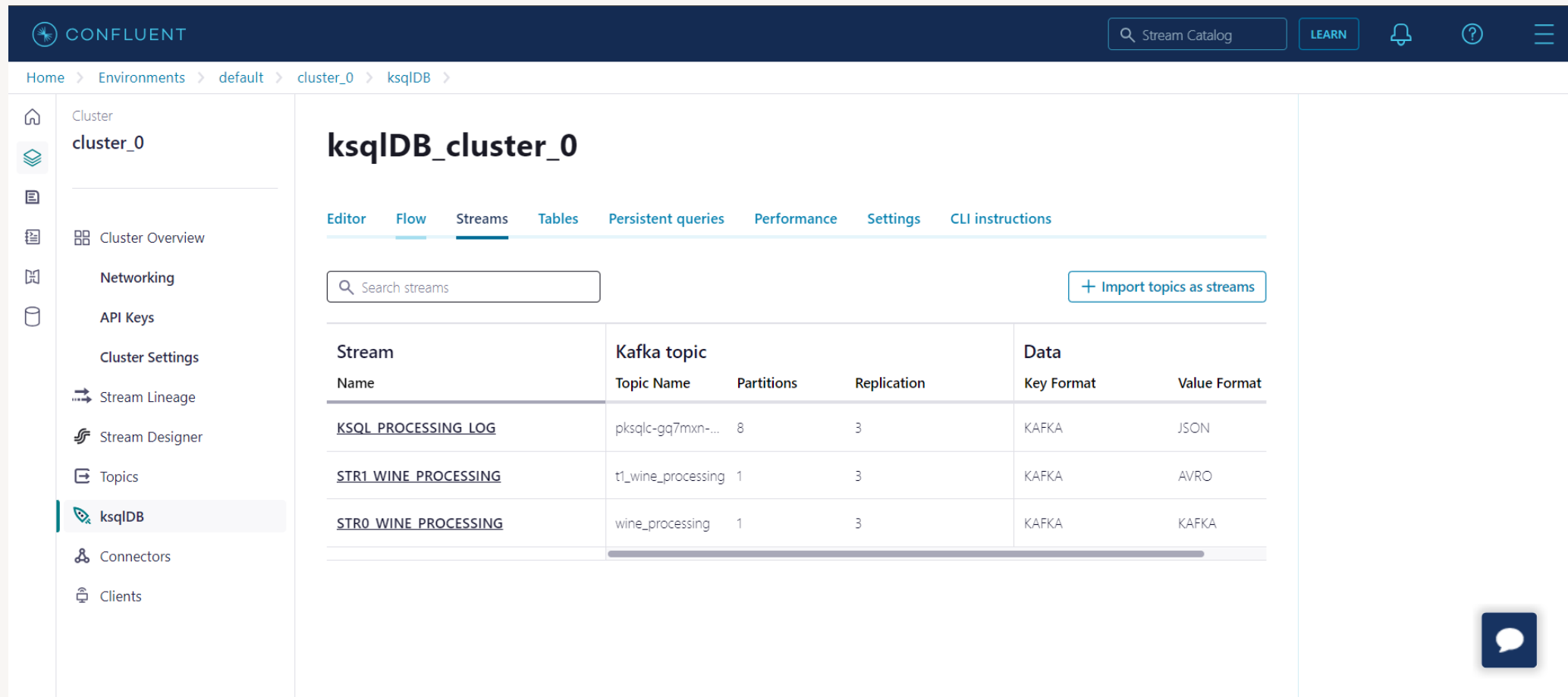
Stream Processing Result

The screenshot displays the Confluent KSQLDB web interface. The top navigation bar includes the Confluent logo, a search bar for the Stream Catalog, and links for LEARN, notifications, help, and a menu. The left sidebar shows the cluster overview for 'cluster_0' and a list of navigation items: Cluster Overview, Networking, API Keys, Cluster Settings, Stream Lineage, Stream Designer, Topics, **ksqlDB** (selected), Connectors, and Clients. The main content area is titled 'ksqlDB_cluster_0' and features tabs for Editor, Flow, Streams, Tables, Persistent queries, Performance, Settings, and CLI instructions. The Editor tab is active, showing a SQL query: `1 SELECT * FROM materialized_view_count_quality EMIT CHANGES;`. Below the query editor, there is a section for 'Add query properties' with a dropdown set to 'auto.offset.reset' and a value of 'Earliest'. A '+Add another field' link is also present. The status 'Running...' is shown with 'Stop' and 'Refresh' buttons. The bottom section displays the 'Data structure' as a TABLE and the 'Total messages' as '--'. A message preview is shown: `{ "WINE_QUALITY": "5.0", "COUNT_QUALITY": 1 }`. At the bottom right, there are checkboxes for JSON and CSV, and a 'Download' button. The right sidebar contains a search bar and a list of 'All available streams and tables': KSQL_PROCESSING_LOG, MATERIALIZED_VIEW_COUNT_QUALITY, STR0_WINE_PROCESSING, and STR1_WINE_PROCESSING.

Dataflow Lineage



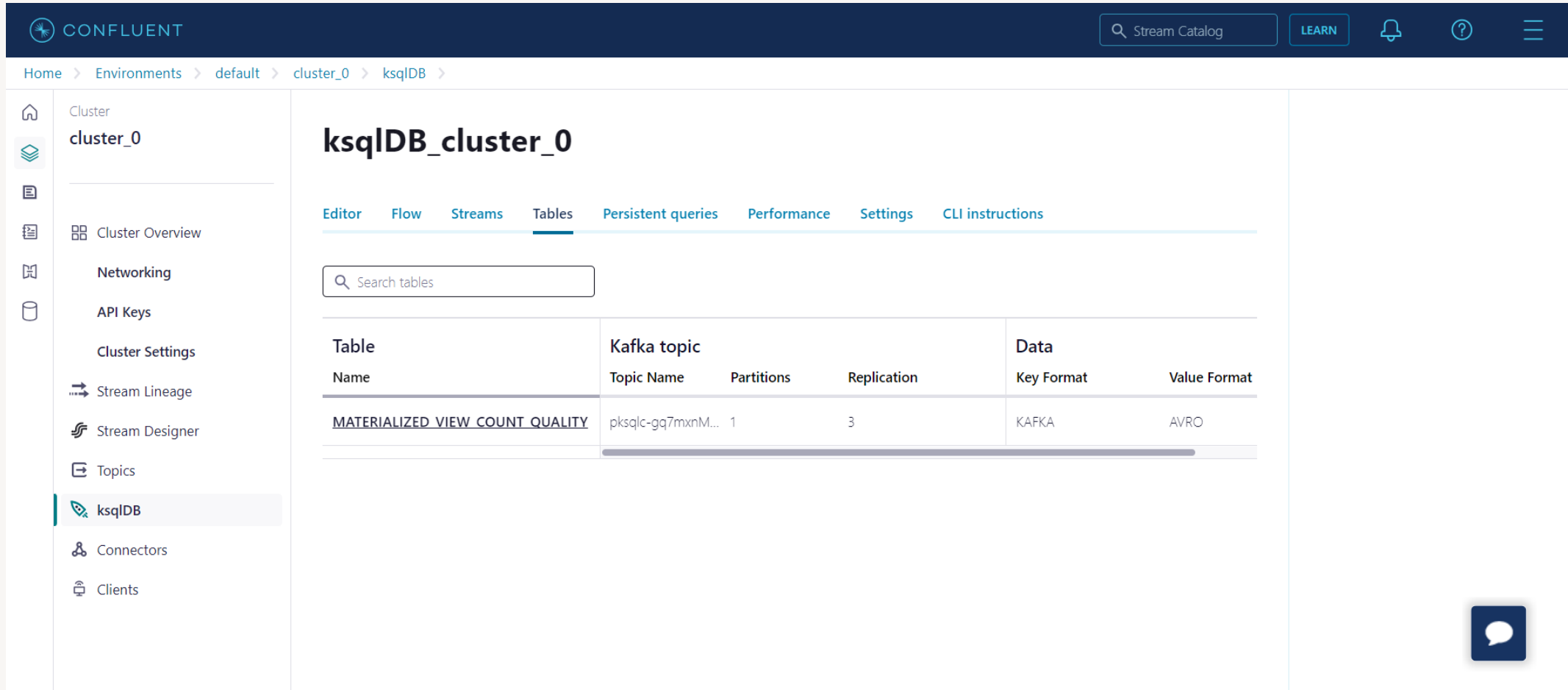
ksqlDB Streams



The screenshot displays the Confluent ksqlDB Streams interface. The top navigation bar includes the Confluent logo, a search bar for 'Stream Catalog', a 'LEARN' button, and notification and help icons. The breadcrumb trail shows the path: Home > Environments > default > cluster_0 > ksqlDB >. The left sidebar lists various cluster management options, with 'ksqlDB' currently selected. The main content area is titled 'ksqlDB_cluster_0' and features a tabbed interface with 'Streams' selected. Below the tabs is a search bar for streams and a button to 'Import topics as streams'. A table lists the configured streams, detailing their names, Kafka topics, partitions, replication factors, and data formats.

Stream Name	Kafka topic	Partitions	Replication	Data Key Format	Data Value Format
KSQL PROCESSING LOG	pksqlc-gq7mxn-...	8	3	KAFKA	JSON
STR1 WINE PROCESSING	t1_wine_processing	1	3	KAFKA	AVRO
STR0 WINE PROCESSING	wine_processing	1	3	KAFKA	KAFKA

ksqlDB Tables



CONFLUENT

Home > Environments > default > cluster_0 > ksqlDB >

Cluster
cluster_0

Cluster Overview

Networking

API Keys

Cluster Settings

Stream Lineage

Stream Designer

Topics

ksqlDB

Connectors

Clients

ksqlDB_cluster_0

Editor Flow Streams **Tables** Persistent queries Performance Settings CLI instructions

Search tables

Table	Kafka topic			Data	
Name	Topic Name	Partitions	Replication	Key Format	Value Format
<u>MATERIALIZED_VIEW_COUNT_QUALITY</u>	pksqlc-gq7mxnM...	1	3	KAFKA	AVRO

ksqlDB Persistent Queries

[Editor](#) [Flow](#) [Streams](#) [Tables](#) [Persistent queries](#) [Performance](#) [Settings](#) [CLI instructions](#)

CTAS_MATERIALIZED_VIEW_COUNT_QUALITY_17

● Running

```
1 CREATE TABLE MATERIALIZED_VIEW_COUNT_QUALITY WITH (CLEANUP_POLICY='compact', KAFKA_TOPIC='pksqlc-gq7mxnMATERIALIZED_VIEW_COUNT_QUALITY', PARTITIONS=1, REPLICAS=3, RETENTION_MS=604800000) AS
2   STR1_WINE_PROCESSING.WINE_QUALITY WINE_QUALITY,
3   COUNT(STR1_WINE_PROCESSING.WINE_QUALITY_VAL) COUNT_QUALITY
4 FROM STR1_WINE_PROCESSING STR1_WINE_PROCESSING
5 GROUP BY STR1_WINE_PROCESSING.WINE_QUALITY
6 EMIT CHANGES;
7
```

ID	CTAS_MATERIALIZED_VIEW_COUNT_QUALITY_17	Source	STR1_WINE_PROCESSING
Sink	MATERIALIZED_VIEW_COUNT_QUALITY		

Metric	Value	Last reported
Messages Per Sec	0	2024-04-21T15:20:29+07:00
Total Messages	2	2024-04-21T15:20:29+07:00

0 errors / sec

Terminate

Explain query

CSAS_STR1_WINE_PROCESSING_15

● Running

```
1 CREATE STREAM STR1_WINE_PROCESSING WITH (CLEANUP_POLICY='delete', KAFKA_TOPIC='t1_wine_processing', PARTITIONS=1, REPLICAS=3, RETENTION_MS=604800000, VALUE_FORMAT='AVRO') AS
2   AS_VALUE(STR0_WINE_PROCESSING.WINE_QUALITY) WINE_QUALITY_VAL,
3   STR0_WINE_PROCESSING.WINE_QUALITY WINE_QUALITY
4 FROM STR0_WINE_PROCESSING STR0_WINE_PROCESSING
5 PARTITION BY STR0_WINE_PROCESSING.WINE_QUALITY
```

Thank you

